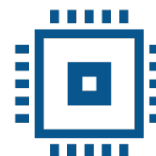




Universitatea *Transilvania* din Braşov
Facultatea de Inginerie Electrică şi Ştiinţa Calculatoarelor
Departamentul Automatică şi Tehnologia Informaţiei



Temă de semestru

P00



Panainte Bogdan Dumitru
TI Grupa 4LF342



Moroşanu Răzvan
TI Grupa 4LF342



Neculcea Sabin
TI Grupa 4LF342



Pricop Andrei Bogdan
TI Grupa 4LF342

BRAŞOV, 2026

Cuprins

1. Motor de poveste interactiva pe baza de blocuri - Evadare din Castelul Bran	3
1.1 Enunt	3
1.2 Prezentare generală a componentelor documentate	3
2. Moroșanu Răzvan: Biblioteca Story.Core	4
2.1 Responsabilități și fișiere	4
2.2 Modelul de date (Poveste.cs)	4
2.2.1 Clasa Poveste — rădăcina grafului	4
2.2.2 Clasa AtributPoveste — proprietățile de stare	4
2.2.3 Clasa BlocPoveste — nodul din graf	5
2.2.4 Clasa Decizie — muchia din graf	5
2.2.5 Clasa Conditie — nod în arborele AST	5
2.2.6 Clasa Efect — modificarea stării	6
2.2.7 Clasa StareJoc — snapshot salvabil	6
2.3 Motorul de joc (MotorPoveste.cs)	6
2.3.1 Câmpuri și proprietăți interne	6
2.3.2 IncarcaPovesteJson(string jsonText)	6
2.3.3 Navigare — MutaLaBloc() și Mergilnapoi()	7
2.3.4 AplicaEfecteSiObțineRedirectionare(Decizie decizie)	7
2.3.5 EvalueazaConditie(Conditie cond) — evaluare recursivă AST	8
2.3.6 Save / Load — ExportaStare() și ImportaStare()	8
2.4 Rezumat API public al motorului	9
3. Panainte Bogdan: Aplicația EvadareBranReader	9
3.1 Responsabilități și fișiere	9
3.2 Arhitectura interfeței grafice	10
3.3 Fluxul de rulare — metoda AfiseazaBlocCurent()	10
3.4 Încărcarea poveștii — DeschidePoveste()	10
3.5 Afișarea imaginilor — AfiseazaImagineBloc()	11
3.6 HUD-ul — ActualizeazaHUD()	11
3.7 Generarea butoanelor de decizii — GenereazaButoaneDecizii()	12
3.8 Handler-ul de decizie — BtnDecizie_Click()	12
3.9 Save / Load stare din meniu	13
3.10 Rezumat metode Form1	13
4. Interacțiunea dintre Story.Core și EvadareBranReader	14
4.1 Principiile de design aplicate	14
5. Neculcea Sabin: Arhitectură și Persistență	15
5.1 Responsabilități și fișiere	15
5.2 Arhitectura de Navigare și Interfața Dinamică	15
5.3 Generarea Procedurală a Editoarelor (Atribute și Blocuri)	16
5.4 Persistența și Gestiunea Resurselor Media (Save / Load)	16
5.5 Rezumat metode cheie Form1.cs	17

6. Pricop Andrei: Decizii, Condiții Logice și Motor de Validare	18
6.1 Responsabilități și arhitectură	18
6.2 Sistemul de Decizii și Efecte (FormDecizie.cs)	19
6.3 Arborele Logic / Sistemul de Condiții AST (FormConditie.cs)	19
6.4 Motorul de Diagnoză și Validare Live (Form1.cs)	20
6.5 Rezumat API intern și funcții dezvoltate	21
7. Rezultate	22
7.1. Aplicația Reader	22
7.2. Aplicația Editor	27
7.3. Validarea poveștii	30
8. Concluzii	31
9. Bibliografie	32

1. Motor de poveste interactiva pe baza de blocuri - Evadare din Castelul Bran

1.1 Enunt

Se cere realizarea în C# a unui sistem pentru povești interactive, bazate pe blocuri de poveste conectate între ele ca un graf orientat.

Proiectul va fi format din două aplicații Windows Forms distincte:

- aplicație pentru editarea poveștii;
- aplicație pentru citirea poveștii.

Povestea este alcătuită din secvențe narrative numite blocuri. Fiecare bloc are un identificator unic, un text descriptiv și o listă de decizii care conduc spre alte blocuri. În funcție de deciziile alese, firul narativ poate diverge, converge și poate conduce la finaluri diferite.

Aplicația trebuie să suporte și un sistem de stare a poveștii, exprimat prin proprietăți numerice configurate din fișierul poveștii. Aceste proprietăți pot controla disponibilitatea deciziilor și pot fi modificate de alegerile jucătorului.

Opțional, fiecare bloc poate avea o imagine de fundal, iar fiecare decizie poate avea o iconiță. Aceste resurse grafice vor fi stocate împreună cu fișierul poveștii.

1.2 Prezentare generală a componentelor documentate

Prezenta documentație acoperă în detaliu contribuțiile a patru membri ai echipei în cadrul proiectului **Evadare din Castelul Bran**: **Moroșanu Răzvan**, **Panainte Bogdan**, **Neculcea Sabin** și **Pricop Andrei**. Proiectul implementează un motor de poveste interactivă în C# / .NET 10, organizat pe trei straturi arhitecturale:

Strat	Proiect	Responsabil
<code>Story.Core</code>	Date + Logică (fără interfață)	Moroșanu Răzvan
<code>EvadareBranReader</code>	Aplicația player (WinForms)	Panainte Bogdan
<code>EvadareBranEditor</code>	Aplicația editor (WinForms)	Neculcea Sabin & Pricop Andrei

Biblioteca `Story.Core` este fundația de care depind celelalte două aplicații — nu conține niciun element de interfață grafică, ceea ce permite testarea sa independentă și reutilizarea în orice context.

Link GitHub: <https://github.com/Bogdy007/Proiect-POO>

2. Moroșanu Răzvan: Biblioteca Story.Core

2.1 Responsabilități și fișiere

Moroșanu Răzvan a proiectat și implementat biblioteca comună **Story.Core**, care conține tot modelul de date și logica motorului de joc. Cele două fișiere livrate sunt:

Fișier	Rol
Story.Core/Models/Poveste.cs	Toate clasele de date cu adnotări JSON
Story.Core/Engine/MotorPoveste.cs	Logica completă de rulare a poveștii

2.2 Modelul de date (Poveste.cs)

Fișierul **Poveste.cs** definește ierarhia de clase care descrie structura completă a unei povești interactive. Fiecare proprietate este adnotată cu `[JsonPropertyName]` pentru a controla numele câmpurilor în fișierul JSON, conform specificației proiectului.

2.2.1 Clasa Poveste — rădăcina grafului

Rol: conține metadatele globale ale poveștii și listele cu atribute și blocuri.

```
public class Poveste
{
    [JsonPropertyName("title")]
    public string Title { get; set; } = "";

    [JsonPropertyName("startBlock")]
    public string StartBlock { get; set; } = "";

    [JsonPropertyName("properties")]
    public List<AtributPoveste> Attributes { get; set; } = new();

    [JsonPropertyName("blocks")]
    public List<BlocPoveste> Blocks { get; set; } = new();
}
```

Câmp StartBlock: identificatorul (string) al blocului de start — motorul de joc îl folosește la inițializare pentru a poziționa jucătorul pe nodul corect din graf. Dacă este gol sau nu există în colecție, se face fallback la primul bloc din listă.

2.2.2 Clasa AtributPoveste — proprietățile de stare

Rol: definește un atribut numeric al jucătorului (ex: viață, suspiciune, galbeni), cu limite, valoare inițială și comportament la atingerea limitelor.

```
public class AtributPoveste
```

```

{
    public string Key { get; set; }          // cheie unică, ex: "player.viata"
    public string HudLabel { get; set; }     // eticheta afișată în HUD
    public int Min { get; set; }             // valoarea minimă
    public int Max { get; set; }             // valoarea maximă
    public int Initial { get; set; }         // valoarea de start
    public bool VisibleInHud { get; set; }   // se afișează în HUD?
    public int HudOrder { get; set; }        // ordinea în HUD
    public string MinBlock { get; set; }     // bloc destinație la atingerea Min
    public string MaxBlock { get; set; }     // bloc destinație la atingerea Max
}

```

Câmpurile MinBlock / MaxBlock: reprezintă un mecanism de redirecționare automată — dacă un efect al unei decizii aduce un atribut la valoarea minimă (ex: viață = 0), motorul sare automat la blocul de moarte, fără logică suplimentară în stratul de prezentare.

2.2.3 Clasa BlocPoveste — nodul din graf

Rol: reprezintă o secvență narativă — un nod în graful orientat al poveștii. Conține textul, imaginea, marcajul de final și lista de decizii disponibile.

```

public class BlocPoveste
{
    public string Id { get; set; }           // identificador unic
    public string Text { get; set; }         // textul narativ
    public string Image { get; set; }        // cale imagine (relativă sau URL)
    public bool IsFinal { get; set; }        // dacă e bloc final
    public List<Decizie> Decisions { get; set; } = new();
}

```

2.2.4 Clasa Decizie — muchia din graf

Rol: reprezintă o opțiune de alegere prezentată jucătorului — muchia dintre două noduri. Include blocul destinație, condiția de vizibilitate și lista de efecte.

```

public class Decizie
{
    public string Text { get; set; }         // textul butonului
    public string TargetBlock { get; set; }  // blocul destinație
    public string Icon { get; set; }         // iconiță opțională
    public Conditie Condition { get; set; } // condiție de afișare (AST)
    public List<Efect> Effects { get; set; } = new();
}

```

2.2.5 Clasa Conditie — nod în arborele AST

Rol: reprezintă un nod în arborele sintactic abstract (AST) al condițiilor logice. Structura este recursivă — un nod AND sau OR poate conține alte noduri Conditie.

```

public class Conditie
{
    public string Type { get; set; }         // "COMPARISON", "AND", "OR"
    public string Property { get; set; }     // atributul vizat (doar COMPARISON)
    public string Operator { get; set; }     // "==", "!=", ">", ">=", "<", "<="
    public int Value { get; set; }           // valoarea de comparat
}

```

```
public List<Conditie> Conditions { get; set; } = new(); // sub-noduri  
}
```

2.2.6 Clasa Efect — modificarea stării

```
public class Efect  
{  
    public string Type { get; set; } // "ADD" sau "SET"  
    public string Property { get; set; } // atributul afectat  
    public int Value { get; set; } // valoarea (poate fi negativă pentru ADD)  
}
```

2.2.7 Clasa StareJoc — snapshot salvabil

Rol: reține starea minimă necesară pentru a relua o sesiune de joc: blocul curent și valorile tuturor atributelor.

```
public class StareJoc  
{  
    public string IdBlocCurent { get; set; }  
    public Dictionary<string, int> Atribute { get; set; } = new();  
}
```

Notă de design: istoricul de navigare (stiva blocurilor vizitate) nu face parte din `StareJoc` și nu se salvează între sesiuni — aceasta este o decizie deliberată, deoarece istoricul este volatil și specific sesiunii curente, nu stării logice a poveștii.

2.3 Motorul de joc (MotorPoveste.cs)

Clasa `MotorPoveste` din namespace-ul `Story.Core.Engine` implementează toată logica de rulare a poveștii, independent de orice interfață grafică. Expune o API simplă și clară pe care o consumă atât `EvadareBranReader`, cât și `EvadareBranEditor`.

2.3.1 Câmpuri și proprietăți interne

```
public Poveste PovesteCurenta { get; private set; }  
public string IdBlocCurent { get; private set; }  
public Dictionary<string, int> StareAtribute { get; private set; }  
  
private Dictionary<string, BlocPoveste> _indexBlocuri; // index O(1)  
private Stack<string> _istoric; // istoric navigare  
public bool PoateMergeInapoi => _istoric.Count > 0;
```

Dictionary vs. List pentru blocuri: Blocurile poveștii sunt indexate după `Id` la momentul încărcării. Accesul ulterior are complexitate **O(1)** față de **O(n)** al unei căutări liniare în listă. Aceasta este importantă în scenariile cu sute de blocuri.

2.3.2 IncarcaPovesteJson(string jsonText)

Deserializează textul JSON în ierarhia de obiecte, construiește indexul blocurilor și inițializează atributele cu valorile de start.

```
public void IncarcaPovesteJson(string jsonText)
{
    PovesteCurenta = JsonSerializer.Deserialize<Poveste>(jsonText);
    // 1. Construim indexul blocurilor pentru acces O(1)
    _indexBlocuri.Clear();
    foreach (var bloc in PovesteCurenta.Blocks)
        if (!string.IsNullOrEmpty(bloc.Id))
            _indexBlocuri[bloc.Id] = bloc;

    // 2. Inițializăm atributele cu valorile de start
    StareAtribute.Clear();
    foreach (var atribut in PovesteCurenta.Attributes)
        StareAtribute[atribut.Key] = atribut.Initial;

    // 3. Poziționăm pe blocul de start (fallback: primul bloc)
    _istoric.Clear();
    IdBlocCurent = _indexBlocuri.ContainsKey(PovesteCurenta.StartBlock)
        ? PovesteCurenta.StartBlock
        : PovesteCurenta.Blocks.FirstOrDefault()?.Id;
}
```

2.3.3 Navigare — MutaLaBloc() și MergiInapoi()

```
public void MutaLaBloc(string idBloc)
{
    if (!string.IsNullOrEmpty(IdBlocCurent))
        _istoric.Push(IdBlocCurent); // salvăm blocul curent în stivă
    IdBlocCurent = idBloc;
}

public bool MergiInapoi()
{
    if (_istoric.Count == 0) return false;
    IdBlocCurent = _istoric.Pop();
    return true;
}
```

2.3.4 AplicaEfecteSiObțineRedirectionare(Decizie decizie)

Aceasta este metoda centrală a motorului. Parcurge efectele deciziei alese, aplică modificările, limitează valorile la intervalul definit și verifică dacă s-a atins o limită care declanșează o redirectionare automată.

```
public string AplicaEfecteSiObțineRedirectionare(Decizie decizie)
{
    foreach (var efect in decizie.Effects)
    {
        var def = PovesteCurenta.Attributes.First(a => a.Key == efect.Property);
        int val = StareAtribute[efect.Property];

        if (efect.Type == "ADD") val += efect.Value;
        else if (efect.Type == "SET") val = efect.Value;
    }
}
```



```

    val = Math.Max(def.Min, Math.Min(def.Max, val));
    StareAtribute[efect.Property] = val;

    // Redirecționare automată la Min sau Max
    if (val == def.Min && !string.IsNullOrEmpty(def.MinBlock))
        return def.MinBlock;
    if (val == def.Max && !string.IsNullOrEmpty(def.MaxBlock))
        return def.MaxBlock;
    }
    return null; // fără redirecționare automată
}

```

Formula de clampare: `Math.Max(min, Math.Min(max, val))` — garantează că niciun atribut nu depășește intervalul [Min, Max] definit, indiferent de valoarea efectului.

2.3.5 EvaluateazaConditie(Conditie cond) — evaluare recursivă AST

Evaluează recursiv arborele de condiții asociat unei decizii. Dacă condiția este null, decizia este mereu vizibilă. Operatorii acceptați sunt: ==, !=, >, >=, <, <=.

```

public bool EvaluateazaConditie(Conditie cond)
{
    if (cond == null) return true; // decizie necondiționată

    if (cond.Type == "COMPARISON")
    {
        int val = StareAtribute[cond.Property];
        return cond.Operator switch {
            "==" => val == cond.Value,
            "!=" => val != cond.Value,
            ">"  => val > cond.Value,
            ">=" => val >= cond.Value,
            "<"  => val < cond.Value,
            "<=" => val <= cond.Value,
            _    => false
        };
    }
    if (cond.Type == "AND")
        return cond.Conditions.All(EvaluateazaConditie); // toate adevărate
    if (cond.Type == "OR")
        return cond.Conditions.Any(EvaluateazaConditie); // cel puțin una

    return false;
}

```

Recursivitate: fiecare nod AND/OR iterează sub-nodurile și apelează `EvaluateazaConditie` recursiv, permițând condiții arbitrar de complexe, imbricate pe mai multe niveluri.

2.3.6 Save / Load — ExportaStare() și ImportaStare()

Permite salvarea și reluarea unei sesiuni de joc. Starea este serializată ca JSON folosind aceleași mecanisme ca și formatul poveștii.

```

public string ExportaStare()
{

```

```

        var stare = new StareJoc {
            IdBlocCurent = this.IdBlocCurent,
            Atribute = new Dictionary<string, int>(this.StareAtribute)
        };
        return JsonSerializer.Serialize(stare, new JsonSerializerOptions { WriteIndented
= true });
    }

    public void ImportaStare(string jsonStare)
    {
        var stare = JsonSerializer.Deserialize<StareJoc>(jsonStare);
        StareAtribute = new Dictionary<string, int>(stare.Atribute ?? new());
        IdBlocCurent = stare.IdBlocCurent;
        _istoric.Clear(); // istoricul nu se restaurează între sesiuni
    }

```

2.4 Rezumat API public al motorului

Metodă	Descriere
<code>IncarcaPovesteJson(string)</code>	Deserializare JSON, index blocuri, inițializare atribute
<code>ObțineBlocCurent()</code>	Returnează structura blocului curent din index
<code>MutaLaBloc(string)</code>	Navighează la bloc, salvează curentul în stivă
<code>MergiInapoi()</code>	Pop din stivă; returnează false dacă stiva e goală
<code>Restart()</code>	Resetează atribute, stivă și bloc la starea inițială
<code>AplicaEfecteSiObțineRedirectionare(Decizie)</code>	Aplică ADD/SET, clampează, returnează bloc redirectionare
<code>EvalueazaConditie(Conditie)</code>	Evaluare recursivă AST COMPARISON/AND/OR
<code>ExportaStare()</code>	Serializează StareJoc ca JSON (save game)
<code>ImportaStare(string)</code>	Restaurează StareJoc din JSON (load game)

3. Panainte Bogdan: Aplicația EvadareBranReader

3.1 Responsabilități și fișiere

În această parte am implementat aplicația player, care permite utilizatorului să încarce și să joace o poveste interactivă. Aceasta consumă direct API-ul `MotorPoveste` al bibliotecii `Story.Core`.

Fișier	Rol
EvadareBranReader/Form1.cs	Logica completă a player-ului
EvadareBranReader/Form1.Designer.cs	Configurarea componentelor UI
EvadareBranReader/Program.cs	Punctul de intrare al aplicației

3.2 Arhitectura interfeței grafice

Fereastra principală (Form1) este organizată în zone funcționale clare:

Componentă	Tip WinForms	Rol
<code>menuStrip1</code>	<i>MenuStrip</i>	Bara de meniu cu Fișier și ◀ Înapoi
<code>lblTitlu</code>	<i>Label</i>	Titlul poveștii (font 14pt, bold, centrat)
<code>lblBlocCurent</code>	<i>Label</i>	ID-ul blocului curent (debug/informativ)
<code>rtbPoveste</code>	<i>RichTextBox</i>	Textul narativ al blocului curent (read-only)
<code>pbImagineBloc</code>	<i>PictureBox</i>	Imaginea de fundal (SizeMode = Zoom)
<code>pnlHUD</code>	<i>Panel</i>	HUD-ul cu valorile atributelor vizibile
<code>flpDecizii</code>	<i>FlowLayoutPanel</i>	Butoanele de decizii (TopDown, AutoScroll)

3.3 Fluxul de rulare — metoda `AfiseazaBlocCurent()`

```
private void AfiseazaBlocCurent()
{
    lblTitlu.Text = _motor.PovesteCurenta?.Title ?? "(poveste fără titlu)";
    lblBlocCurent.Text = "Bloc: " + (_motor.IdBlocCurent ?? "?");
    inapoiToolStripMenuItem.Enabled = _motor.PoateMergeInapoi;
}
```

```

var bloc = _motor.ObțineBlocCurent();
rtbPoveste.Text = bloc.Text;
AfiseazaImagineBloc(bloc); // imagine locală sau URL
ActualizeazaHUD(); // valorile atributelor

if (bloc.IsFinal)
    AfiseazaFinal(); // mesaj + buton restart
else
    GenereazaButoaneDecizii(bloc); // filtrare + generare butoane
}

```

3.4 Încărcarea poveștii — DeschidePoveste()

```

private void DeschidePoveste()
{
    // Filtrare: *.zip;*.json
    if (extensie == ".zip")
    {
        // Extrage într-un folder temporar unic (Guid)
        _folderResurse = Path.Combine(Path.GetTempPath(),
            "EvadareBranReader_" + Guid.NewGuid().ToString("N"));
        Directory.CreateDirectory(_folderResurse);
        ZipFile.ExtractToDirectory(ofd.FileName, _folderResurse);
        jsonText = File.ReadAllText(Path.Combine(_folderResurse, "story.json"));
    }
    else if (extensie == ".json")
    {
        // Folderul de resurse = directorul fișierului JSON
        _folderResurse = Path.GetDirectoryName(ofd.FileName);
        jsonText = File.ReadAllText(ofd.FileName);
    }
    _motor.IncarcaPovesteJson(jsonText);
    AfiseazaBlocCurent();
}

```

Gestionarea folderului temporar: folderul extras din ZIP este șters automat la închiderea aplicației (`Form1_FormClosing`) sau la deschiderea unei noi povești (`CurataFolderResurse`). Identificatorul GUID din nume previne coliziunile dacă aplicația rulează simultan în mai multe instanțe.

3.5 Afișarea imaginilor — AfiseazaImagineBloc()

Suportă două surse de imagini: fișiere locale (din folderul de resurse) și URL-uri HTTP/HTTPS (încărcate asincron).

```

private void AfiseazaImagineBloc(BlocPoveste bloc)
{
    if (bloc.Image.StartsWith("http://") || bloc.Image.StartsWith("https://"))
    {
        // Asincron - PictureBox.LoadAsync() este thread-safe
        pbImagineBloc.LoadAsync(bloc.Image);
    }
    else if (!string.IsNullOrEmpty(_folderResurse))
    {
        string cale = Path.Combine(_folderResurse, bloc.Image);
    }
}

```

```
        if (File.Exists(cale))
        {
            // FileStream + Image.FromStream evită blocarea fișierului
            using (var fs = new FileStream(cale, FileMode.Open, FileAccess.Read))
                pbImagineBloc.Image = Image.FromStream(fs);
        }
    }
}
```

De ce FileStream + Image.FromStream? Încărcarea directă cu `Image.FromFile()` lasă fișierul blocat cât timp imaginea este afișată. Folosind un `FileStream` în bloc `using`, fișierul este eliberat imediat după citire.

3.6 HUD-ul — ActualizeazaHUD()

```
private void ActualizeazaHUD()
{
    pnlHUD.Controls.Clear();
    int y = 10;

    var vizibile = _motor.PovesteCurenta.Attributes
        .Where(a => a.VisibleInHud)
        .OrderBy(a => a.HudOrder);

    foreach (var attr in vizibile)
    {
        int val = _motor.StareAtribute[attr.Key];
        Label lbl = new Label {
            Text = $"{attr.HudLabel}: {val} / {attr.Max}",
            Font = new Font("Segoe UI", 10, FontStyle.Bold),
            Location = new Point(10, y),
            AutoSize = true
        };
        pnlHUD.Controls.Add(lbl);
        y += 25;
    }
}
```

3.7 Generarea butoanelor de decizii — GenereazaButoaneDecizii()

```
private void GenereazaButoaneDecizii(BlocPoveste bloc)
{
    flpDecizii.Controls.Clear();

    foreach (var decizie in bloc.Decisions)
    {
        // Filtrăm deciziile după condiția AST
        if (!_motor.EvalueazaConditie(decizie.Condition)) continue;

        Button btn = new Button {
            Text = decizie.Text,
            Width = 540, Height = 40,
            TextAlign = ContentAlignment.MiddleLeft,
            Tag = decizie // stocăm referința la decizie pe buton
        };
    }
}
```

```
};  
btn.Click += BtnDecizie_Click;  
flpDecizii.Controls.Add(btn);  
}  
}
```

3.8 Handler-ul de decizie — BtnDecizie_Click()

Aplică efectele deciziei selectate prin motor și realizează tranziția la blocul următor.

```
private void BtnDecizie_Click(object sender, EventArgs e)  
{  
    Decizie decizie = (sender as Button).Tag as Decizie;  
  
    // Aplicăm efectele (ADD/SET) și verificăm redirectionarea automată  
    string redirect = _motor.AplicaEfecteSiObțineRedirectionare(decizie);  
  
    // Dacă un atribut a atins limita, mergem la blocul de redirectionare  
    _motor.MutaLaBloc(redirect ?? decizie.TargetBlock);  
  
    AfiseazaBlocCurent();  
}
```

Operatorul ??: `redirect ?? decizie.TargetBlock` — dacă motorul returnează un bloc de redirectionare (atribut la limită), mergem acolo; altfel mergem la blocul normal al deciziei.

3.9 Save / Load stare din meniu

Meniul Fișier expune opțiunile de salvare și încărcare a stării jocului, delegând complet logica motorului.

```
// SALVARE  
private void salveazaStareToolStripMenuItem_Click(...)  
{  
    using (SaveFileDialog sfd = new SaveFileDialog())  
    {  
        sfd.Filter = "Stare joc (*.json)|*.json";  
        if (sfd.ShowDialog() == DialogResult.OK)  
            File.WriteAllText(sfd.FileName, _motor.ExportaStare());  
    }  
}  
  
// ÎNCĂRCARE  
private void incarcaStareToolStripMenuItem_Click(...)  
{  
    using (OpenFileDialog ofd = new OpenFileDialog())  
    {  
        ofd.Filter = "Stare joc (*.json)|*.json";  
        if (ofd.ShowDialog() == DialogResult.OK)  
        {  
            _motor.ImportaStare(File.ReadAllText(ofd.FileName));  
            AfiseazaBlocCurent();  
        }  
    }  
}
```

3.10 Rezumat metode Form1

Metodă	Descriere
<code>DeschidePoveste()</code>	Deschide ZIP sau JSON, extrage resurse, inițializează motorul
<code>AfiseazaBlocCurent()</code>	Actualizează toate zonele UI în funcție de starea motorului
<code>AfiseazaImagineBloc(bloc)</code>	Încarcă imaginea local (FileStream) sau asincron (URL)
<code>ActualizeazaHUD()</code>	Regenerează dinamic etichetele de atribute vizibile
<code>GenereazaButoaneDecizii(bloc)</code>	Generează butoane filtrate după condițiile AST
<code>BtnDecizie_Click(...)</code>	Aplică efectele și realizează tranziția de bloc
<code>AfiseazaFinal()</code>	Afișează mesajul de final și butonul de restart
<code>CurataFolderResurse()</code>	Șterge folderul temporar creat din ZIP
<code>salveazaStareToolStripMenuItem_Click</code>	Exportă starea curentă ca JSON (SaveFileDialog)
<code>incarcaStareToolStripMenuItem_Click</code>	Importă o stare salvată din JSON (OpenFileDialog)
<code>inapoiToolStripMenuItem_Click</code>	Apelează <code>MergiInapoi()</code> și reafișează blocul anterior

4. Interacțiunea dintre Story.Core și EvadareBranReader

Separarea clară dintre bibliotecă și interfață permite ca schimbările în UI să nu afecteze logica de joc și invers. Diagrama de mai jos ilustrează fluxul complet al unei acțiuni de joc:

```

Utilizator apasă un buton de decizie
    |
    v
BtnDecizie_Click() [Form1 - Persoana 2]
    |
    
```

```

1]      |---> AplicaEfecteSiObțineRedirectionare(decizie) [MotorPoveste - Persoana
      |
      |      |-- aplică ADD/SET pe StareAtribute
      |      |-- clampează la [Min, Max]
      |      '-- returnează bloc redirectionare (sau null)
      |
      |---> MutaLaBloc(idBloc) [MotorPoveste - Persoana 1]
      |      '-- Push pe _istoric, actualizează IdBlocCurent
      |
      v
AfiseazaBlocCurent() [Form1 - Persoana 2]
|-- ObțineBlocCurent() -> BlocPoveste [MotorPoveste]
|-- EvalueazaConditie() pentru fiecare decizie [MotorPoveste]
|-- Actualizează UI: text, imagine, HUD, butoane
'-- Dacă IsFinal: AfiseazaFinal()
    
```

4.1 Principiile de design aplicate

- **Separarea responsabilităților (SoC):** logica de joc este complet izolată în Story.Core; Form1 nu conține nicio logică de evaluare a condițiilor sau de aplicare a efectelor.
- **Principiul deschis/închis:** adăugarea unui nou tip de efect sau condiție necesită modificarea doar a Story.Core, nu a aplicațiilor consumatoare.
- **Acces O(1) prin Dictionary:** indexul de blocuri evită parcurgerea secvențială la fiecare tranziție.
- **Gestionarea resurselor:** folderul temporar și imaginile sunt eliberate corect prin blocuri `using` și prin `Dispose()`.
- **Thread safety:** `PictureBox.LoadAsync()` este apelat de pe thread-ul UI, evitând probleme de concurență la descărcarea imaginilor de pe net.

5. Neculcea Sabin: Arhitectură și Persistență

5.1 Responsabilități și fișiere

Neculcea Sabin a implementat infrastructura de bază a aplicației EvadareBranEditor, optând pentru o arhitectură de interfață bazată pe o singură fereastră reactivă. Responsabilitățile sale au acoperit construirea componentei TreeView pentru navigarea ierarhică a poveștii, generarea procedurală a formularelor pentru operațiunile CRUD (atribute și blocuri narrative), gestiunea resurselor media și persistența pachetelor de tip ZIP.

Fișier	Rol arhitectural

EvadareBranEditor/Form1.cs	Interfața centrală (navigare <code>TreeView</code> și panou reactiv <code>pnlEditorCentral</code>). Gestionează crearea dinamică a UI-ului, împachetarea arhivelor <code>.zip</code> și parsarea JSON.
EvadareBranEditor/Program.cs	Punctul de intrare în aplicație și configurarea inițială a firelor de execuție UI.

5.2 Arhitectura de Navigare și Interfața Dinamică

Pentru a evita fragmentarea aplicației și consumul excesiv de memorie prin instanțierea de ferestre multiple, interfața a fost divizată în două zone funcționale interconectate: o componentă `TreeView` poziționată în stânga și un panou central (`pnlEditorCentral`) care se redesenează la fiecare selecție.

Mecanismul de rutare (`tvPoveste_AfterSelect`): Când utilizatorul selectează un nod din arbore, panoul central este curățat prin `Controls.Clear()`, iar aplicația determină ce interfață trebuie generată pe baza obiectului stocat în proprietatea `Tag` a nodului curent:

```
if (e.Node?.Tag is string tag)
{
    if (tag == TAG_ROOT) AfiseazaEditorMetadate();
    else if (tag == TAG_CAT_ATTRIBUTE) AfiseazaCategorieAttribute();
    else if (tag == TAG_CAT_BLOCURI) AfiseazaCategorieBlocuri();
}
else if (e.Node?.Tag is AtributPoveste atribut)
{
    AfiseazaEditorAtribut(atribut, e.Node); // Generează interfața atributului
}
// Logica continuă similar pentru BlocPoveste
```

5.3 Generarea Procedurală a Editoarelor (Atribute și Blocuri)

În loc să se instanțieze controale statice direct din designer, formularele de editare a entităților sunt construite matematic și dinamic la momentul execuției.

- **Editorul de Atribute (`AfiseazaEditorAtribut`):** Implementează manipularea completă a proprietăților stării de joc. Instanțiază dinamic controale `NumericUpDown` pentru limitarea matematică a valorilor (`Min`, `Max`, `Initial`), un `CheckBox` pentru vizibilitatea în HUD și controale `ComboBox` populate live cu ID-urile tuturor blocurilor existente pentru setarea parametrilor de redirecționare automată (`MinBlock` și `MaxBlock`).
- **Editorul de Blocuri (`AfiseazaEditorBloc`):** Permite scrierea textului narativ, atașarea de imagini (cu funcție de previzualizare) și marcarea stării finale a blocului (`IsFinal`). Interconectarea cu modulul de decizii dezvoltat de Pricop Andrei se realizează prin transmiterea listelor curente de identificatori pentru a asigura o validare facilă:

```
// Interconectarea cu modulul de logică (Persoana 4)
var idBlocuri = _povesteCurenta.Blocks.Select(b => b.Id).ToList();
var attribute = _povesteCurenta.Attributes.Select(a => a.Key).ToList();
using (FormDecizie frm = new FormDecizie(idBlocuri, attribute))
{
    if (frm.ShowDialog() == DialogResult.OK) {
        bloc.Decisions.Add(frm.DecizieRezultat);
        ReincarcaListaDecizii(lbDecizii, bloc);
    }
}
```

5.4 Persistența și Gestiunea Resurselor Media (Save / Load)

O funcționalitate critică a editorului este împachetarea poveștii (fișier `story.json`) împreună cu resursele sale media într-un singur fișier distribuibil de tip `.zip`.

Sistemul de urmărire a imaginilor: S-a implementat un mecanism de *tracking* eficient. Imaginile noi selectate de utilizator de pe disc sunt reținute într-un dicționar temporar (`_imaginiNoi`). La salvarea proiectului (`SalveazaPoveste`), motorul execută următorii pași:

1. Creează un director temporar folosind un `Guid` unic pentru a evita coliziunile.
2. Apelează funcția `CopiazalImaginiReferite()`, care scanează întregul graf al poveștii și copiază fizic în subfolderul `images/` imaginile din dicționarul de imagini noi sau cele extrase din folderul sursă.
3. Serializează obiectul `Poveste` utilizând un encoder relaxat pentru a permite stocarea de caractere diacritice și linkuri externe netrunchiate:

```
var options = new JsonSerializerOptions {
    WriteIndented = true,
    Encoder = System.Text.Encodings.Web.JavaScriptEncoder.UnsafeRelaxedJsonEscaping
};
```

4. Creează arhiva finală prin `ZipFile.CreateFromDirectory()` și șterge automat folderul temporar.

Încărcarea previzualizării imaginilor în panoul UI se realizează asincron (`LoadAsync`) pentru URL-uri externe sau prin `FileStream` pentru fișierele locale. Această decizie arhitecturală garantează că fișierele locale nu sunt blocate de sistemul de operare și pot fi suprascrise la salvare.

5.5 Rezumat metode cheie `Form1.cs`

Metodă / Operațiune	Scop și Funcționalitate

CreazaPovesteGoala()	Resetează starea editorului, inițializând o structură de date nouă și curățând dicționarul <code>_imaginiNoi</code> și folderele temporare.
ActualizeazaTreeView()	Reconstruiește structura de noduri ierarhice pe baza memoriei curente și forțează rularea algoritmului de validare live.
AfiseazaEditorAtribut() / AfiseazaEditorBloc()	Gestionează randarea procedurală a interfeței grafice pentru modificarea entităților și aplicarea mutațiilor direct în colecțiile din memorie (CRUD).
IncarcaPreviewImagine()	Metodă hibridă (<code>async</code> pentru URL-uri, <code>FileStream</code> pentru căi locale) responsabilă de afișarea sigură a graficii în <code>PictureBox</code> .
CopiazalmaginiReferite()	Sistem inteligent de rezoluție a fișierelor media, garantând copierea corectă a resurselor locale în subdirectorul <code>images/</code> pentru pachetul <code>.zip</code> .
PromptText()	Dialog UI instanțiat procedural, folosit pentru colectarea primară a cheilor și ID-urilor unice la instanțierea entităților noi.

6. Pricop Andrei: Decizii, Condiții Logice și Motor de Validare

6.1 Responsabilități și arhitectură

Pricop Andrei a implementat subsistemele avansate ale aplicației EvadareBranEditor, care permit crearea deciziilor, generarea arborilor logici pentru condiții și validarea în timp real a poveștilor. Responsabilitățile sale s-au împărțit în trei direcții arhitecturale majore:

1. **Controlul fluxului și mutațiilor de stare** (gestionarea deciziilor și a efectelor matematice asupra atributelor).

2. **Evaluarea logică** (proiectarea unui editor vizual pentru Arbori Sintactici Abstracte - AST).
3. **Integritatea datelor** (dezvoltarea unui sistem de validare *live* și a unui algoritm de parcurgere a grafurilor pentru detectarea erorilor).

Fișier	Rol arhitectural
EvadareBranEditor/FormDecizie.cs	Formular generat dinamic pentru definirea tranzițiilor (muchiiilor) și a efectelor (ADD/SET).
EvadareBranEditor/FormConditie.cs	Editor vizual recursiv pentru arborele logic al deciziilor (Abstract Syntax Tree).
EvadareBranEditor/Form1.cs	Sistemul de validare în timp real și algoritmul BFS pentru accesibilitate.

6.2 Sistemul de Decizii și Efecte (**FormDecizie.cs**)

Clasa **FormDecizie** gestionează instanțierea și editarea obiectelor de tip **Decizie**. Pentru a oferi un grad maxim de control asupra randării interfeței, elementele vizuale (controalele) nu au fost generate prin designer-ul Visual Studio, ci instanțiate pur matematic prin cod, în metoda **ConstruiesteInterfata()**.

1. Încapsulare și Data Binding:

Transferul de date între arborele principal al poveștii și formular se face respectând principiul încapsulării, utilizând un *setter intelligent* care declanșează actualizarea interfeței:

```
[Browsable(false)]
[DesignerSerializationVisibility(DesignerSerializationVisibility.Hidden)]
public Decizie DecizieRezultat {
    get => _decizie;
    set { _decizie = value; IncarcaDate(); }
}
private Decizie _decizie;
```

Adnotările previn serializarea accidentală a clasei complexe în designer-ul WinForms. De asemenea, a fost utilizată **înlanțuirea constructorilor** (`this(...)`) pentru a asigura inițializarea corectă a listelor chiar și la instanțierea fără parametri, prevenind `NullReferenceException`.

2. Gestionarea Mutațiilor (Efecte):

Pentru definirea efectelor asupra atributelor, a fost implementat un control `DataGridView` cu trei coloane (Tip, Atribut, Valoare). Salvarea datelor aplică tehnici de *Input Validation* și *Guard Clauses*:

```
int valoare = 0;
// TryParse asigură stabilitatea programului; valorile invalide devin 0 silențios
int.TryParse(row.Cells[2].Value?.ToString(), out valoare);

_decizie.Effects.Add(new Efect
{
    Type = row.Cells[0].Value.ToString().ToUpper(), // Standardizare ADD/SET
    Property = row.Cells[1].Value.ToString(),
    Value = valoare
});
```

6.3 Arborele Logic / Sistemul de Condiții AST (`FormConditie.cs`)

Pentru a permite autorilor să creeze condiții de vizibilitate complexe (ex: `(Bani > 50 ȘI AiCheia == 1) SAU Energie > 80`), a fost dezvoltat un editor vizual bazat pe componenta `TreeView`. Aceasta reprezintă interfața grafică a clasei `Conditie` din `Story.Core`.

Deoarece adâncimea arborelui logic este teoretic infinită, sincronizarea dintre date și interfață a necesitat implementarea a două **metode recursive**:

1. Deserializarea în interfață (Încărcarea):

Metoda `IncarcaConditieInArbore` parcurge obiectul `Conditie` venit din motor și construiește nodurile vizuale corespunzătoare (`COMPARISON`, `AND`, `OR`), coborând recursiv pe fiecare ramură.

2. Serializarea din interfață (Salvarea):

La apăsarea butonului de salvare, interfața este recitită de jos în sus, reambalând datele vizuale într-o structură pe care biblioteca `Story.Core` o poate procesa:

```
private Conditie ConstruiesteConditieDinArbore(TreeNode nodVizual)
{
    // Inițializare nod curent pe baza Tag-ului vizual
    Conditie conditieCurenta = extrageDateNod(nodVizual);

    // Auto-apelare pentru fiecare sub-ramură
    foreach (TreeNode copil in nodVizual.Nodes)
    {
        conditieCurenta.Conditions.Add(ConstruiesteConditieDinArbore(copil));
    }
}
```

```
}  
    return conditieCurenta;  
}
```

6.4 Motorul de Diagnoză și Validare Live (Form1.cs)

Orice modificare a structurii poveștii în Editor apelează metoda `RuleazaValidareInJurnal()`. Aceasta aplică o suită de reguli stricte pentru a garanta că fișierul `.zip` final nu va bloca aplicația Player.

1. Scanarea Integrității Structurale:

- **Referințe orfane:** Se verifică dacă fiecare decizie are un `TargetBlock` ce există fizic în dicționarul de blocuri.
- **Redirecționări automate:** Se validează existența blocurilor specificate în `MinBlock` și `MaxBlock` pentru fiecare atribut.
- **Consecvența referințelor de stare:** Efectele (`ADD/SET`) și Condițiile (`COMPARISON`) sunt validate pentru a se asigura că referențiază chei de attribute valide.
- **Matematica atributelor:** Se impune matematic condiția `Min <= Initial <= Max`.
- **Resurse media:** Se scanează proprietățile de imagine pentru a garanta formatul corect (`.jpg`, `.png`, `.bmp`, `.gif`) sau un URL valid.

2. Algoritmul de Accesibilitate (BFS):

Pentru a asigura navigabilitatea grafului și a detecta blocurile izolate (la care jucătorul nu poate ajunge niciodată prin nicio decizie), a fost implementat un algoritm de căutare în lățime (*Breadth-First Search*):

```
private void VerificaAccesibilitate(List<string> erori)  
{  
    Queue<string> coada = new Queue<string>();  
    HashSet<string> vizitate = new HashSet<string>();  
  
    // Punctul de intrare în graf  
    if (_povesteCurenta.Blocks.Any(b => b.Id == _povesteCurenta.StartBlock))  
    {  
        coada.Enqueue(_povesteCurenta.StartBlock);  
        vizitate.Add(_povesteCurenta.StartBlock);  
    }  
  
    // Parcurgerea BFS  
    while (coada.Count > 0)  
    {  
        string idCurent = coada.Dequeue();  
        var bloc = _povesteCurenta.Blocks.First(b => b.Id == idCurent);  
  
        foreach (var decizie in bloc.Decisions)  
        {  
            if (decizie.TargetBlock != null && !vizitate.Contains(decizie.TargetBlock))  
            {
```

```

        vizitate.Add(decizie.TargetBlock);
        coada.Enqueue(decizie.TargetBlock);
    }
}

// Identificarea insulelor
var toateBlocurile = _povesteCurenta.Blocks.Select(b => b.Id);
var inaccesibile = toateBlocurile.Except(vizitate);

foreach(var izolat in inaccesibile)
{
    erori.Add($"• Bloc inaccesibil: '{izolat}'");
}
}

```

6.5 Rezumat API intern și funcții dezvoltate

Metodă / Tehnologie	Scop și Funcționalitate
ConstruiesteInterfata()	Generare procedurală (cod) a layout-ului pentru FormDecizie , cu utilizarea de <i>Lambda Expressions</i> pentru delegarea evenimentelor de click.
ConstruiesteConditieDinArbore()	Prelucrare algoritmică recursivă a obiectelor TreeNode pentru asamblarea structurii AST.
IncarcaConditieInArbore()	Parcurea obiectului serializabil Conditie pentru randarea vizuală corespunzătoare gradului de adâncime al logicii.
ValideazaPoveste()	Funcția master de diagnostic; returnează lista de mesaje afișată live în lbJurnal .
VerificaAccesibilitate()	Parcurea componentelor conexe ale grafului (BFS) pentru optimizarea și corectarea schemei narrative a autorului.

7. Rezultate

Pentru a demonstra funcționalitățile aplicației Reader am parcurs un scenariu complet de joc folosind povestea de test „*Evadarea din Bran*”.

7.1. Aplicația Reader

7.1.1. La pornire, aplicația nu are încă nicio poveste încărcată. Titlul afișează „(nicio poveste încărcată)”, zona de decizii și HUD-ul sunt goale, iar din meniul *Fișier* avem la dispoziție opțiunile de deschidere, salvare și restart.

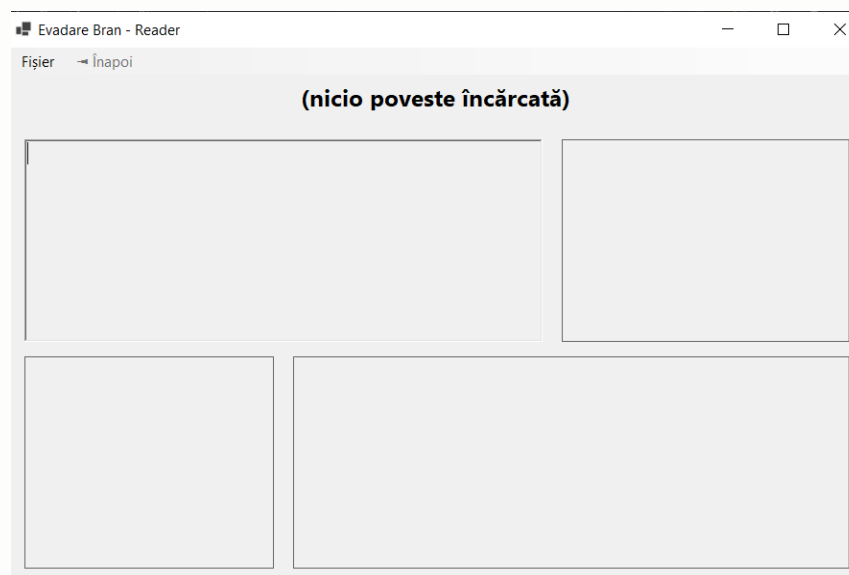


Figura 7. 1 — Fereastra aplicației la pornire.

7.1.2. Din *Fișier* → *Deschide poveste...* se deschide un dialog care acceptă atât arhive `.zip` (poveste împreună cu imaginile ei), cât și fișiere `.json` simple. În cazul arhivei, aplicația o dezarchivează într-un folder temporar unic și citește din el fișierul `story.json`.

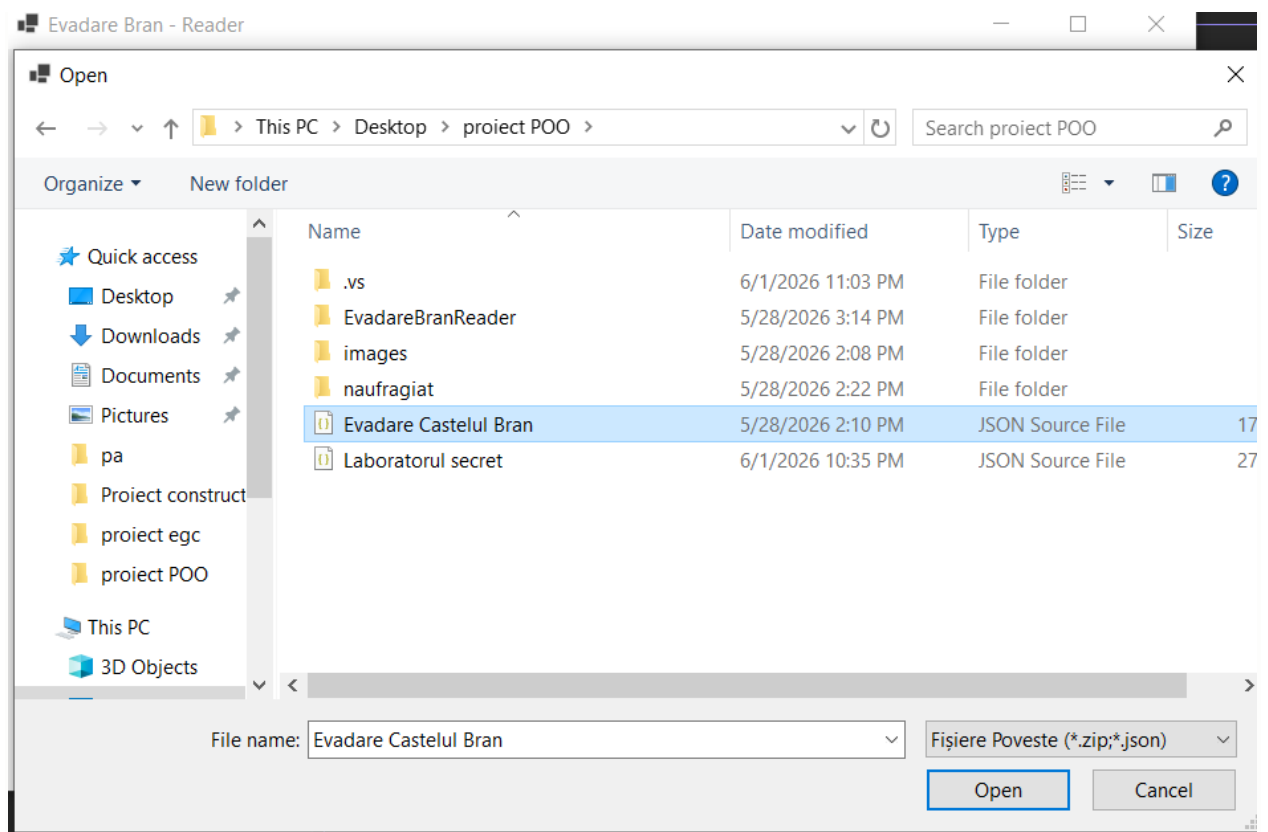


Figura7. 2 — Dialogul de deschidere a unei povești.

7.1.3. După încărcare se afișează primul bloc al poveștii. În stânga apare textul narativ, în dreapta imaginea aferentă blocului, în partea de jos HUD-ul cu atributele jucătorului, iar deciziile disponibile sub formă de butoane. Toată interfața este construită dinamic, pe baza datelor din poveste.



Figura 7.3 — Afișarea unui bloc de poveste.

7.1.4. HUD-ul arată doar atributele marcate ca vizibile, în ordinea stabilită de autor. Fiecare atribut (viață, energie, bani) este afișat sub forma `valoare curentă / valoare maximă` și se actualizează automat după fiecare decizie.

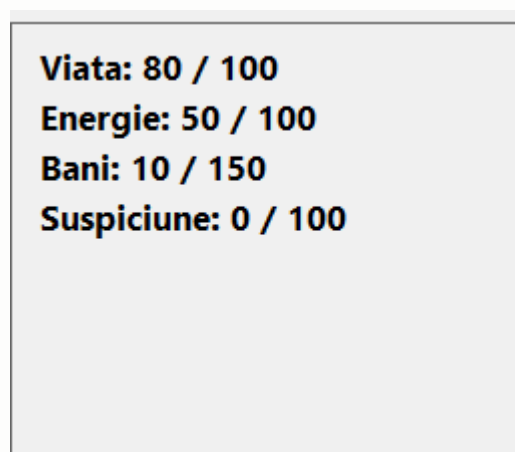


Figura 7.4 — HUD-ul cu atributele jucătorului.

7.1.5. Anumite decizii apar doar dacă o condiție este îndeplinită. De exemplu, opțiunea „Deschide ușa cu cheia” devine vizibilă abia după ce jucătorul a obținut cheia; cât timp condiția este falsă, butonul lipsește complet din listă.

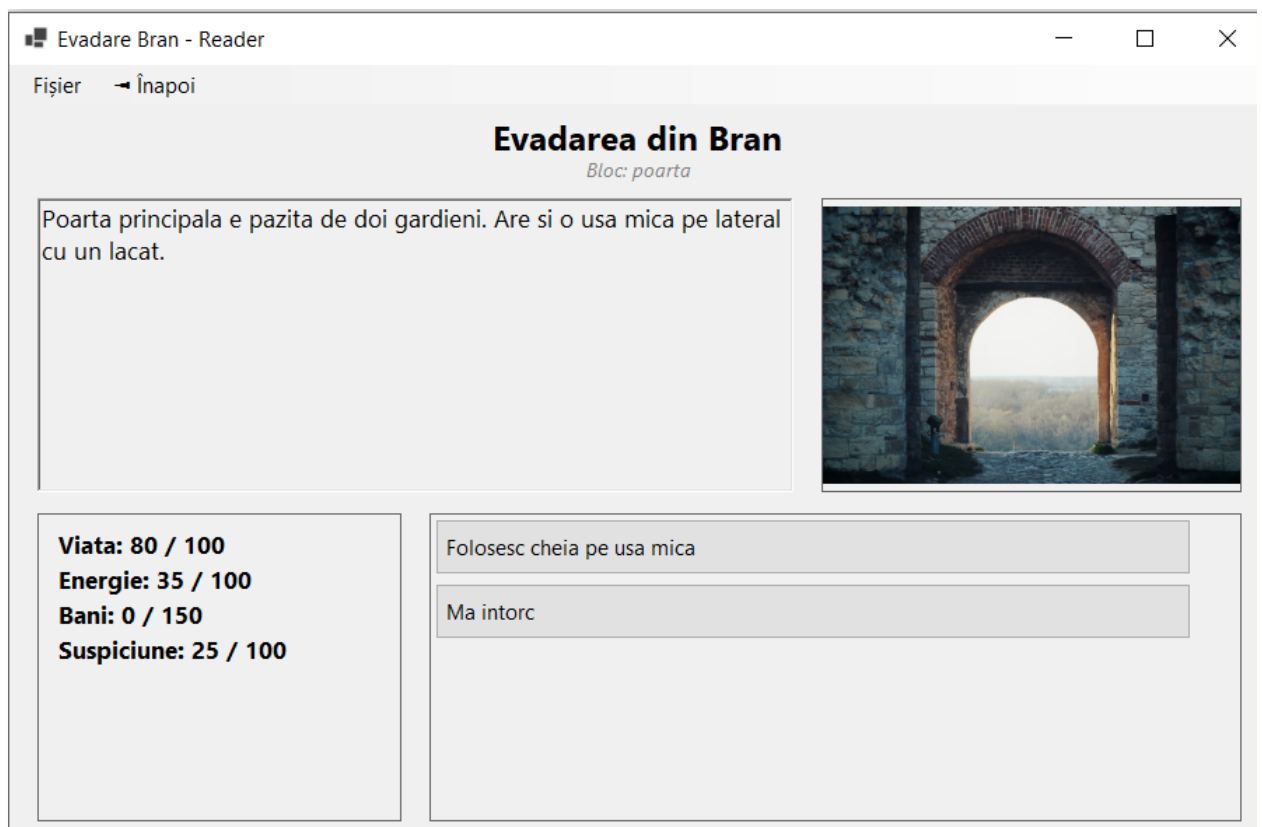


Figura 7.5 – Decizie condiționată de starea jucătorului.

7.1.6 Prin **Fișier** → **Salvează stare...**, progresul curent (blocul în care ne aflăm împreună cu valorile atributelor) este exportat într-un fișier .json. Un mesaj confirmă salvarea reușită.

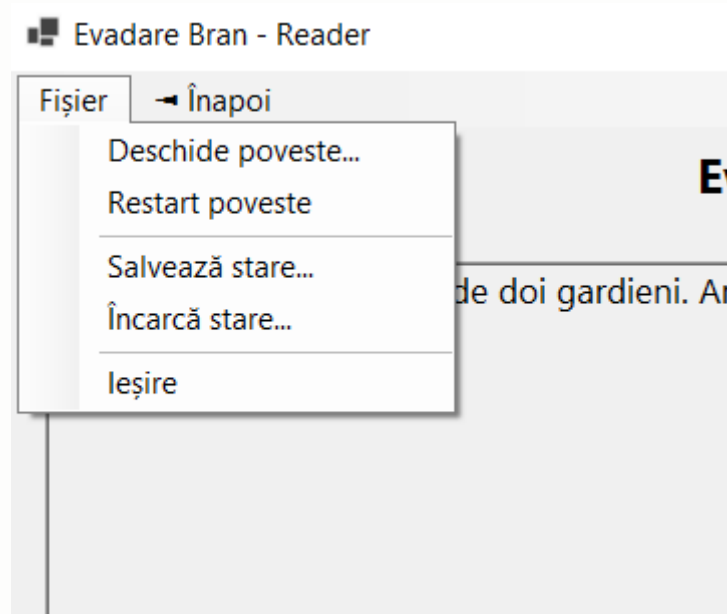


Figura 7.6 – Salvarea stării jocului.

7.1.7 O stare salvată anterior poate fi reîncărcată, jocul revenind exact la blocul respectiv cu aceleași valori ale atributelor. În plus, butonul „◀ Înapoi” devine activ după primele decizii și permite revenirea la blocurile deja vizitate.

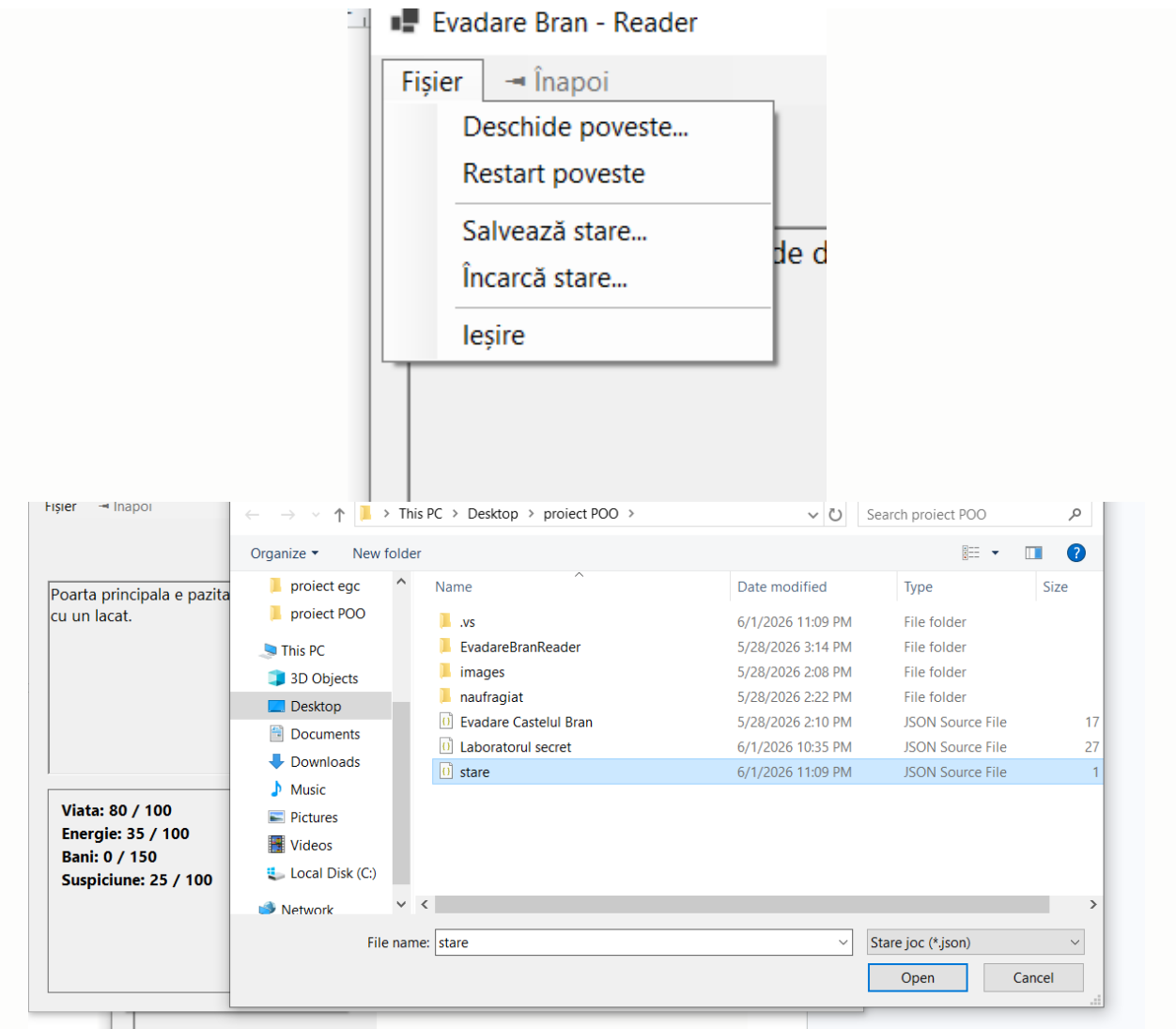


Figura 7.7 – Încărcarea stării și navigarea înapoi.

7.1.8. Într-un bloc final se afișează mesajul „*** SFÂRȘITUL POVEȘTII ***” împreună cu un buton de *Restart*, care readuce jocul la blocul de start și resetează atributele.

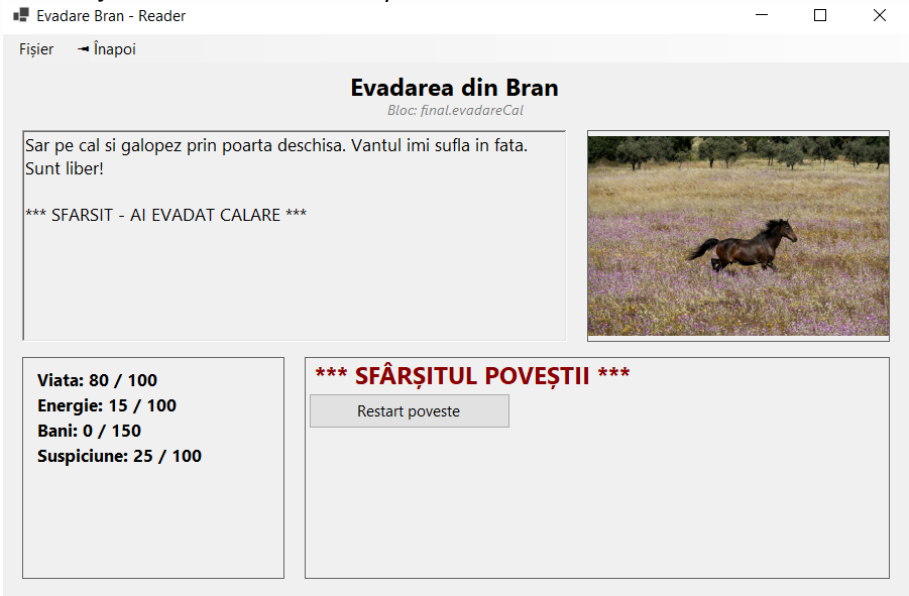


Figura 7.8 — Ecranul de final al poveștii.

7.2. Aplicația Editor

7.2.1. La pornire, Editorul afișează o poveste nouă, goală. În stânga se află arborele poveștii (TreeView) cu cele două categorii — „Proprietăți (atribute)” și „Blocuri narative” — iar zona centrală este liberă până la selectarea unui element. În partea de jos se află jurnalul de validare.

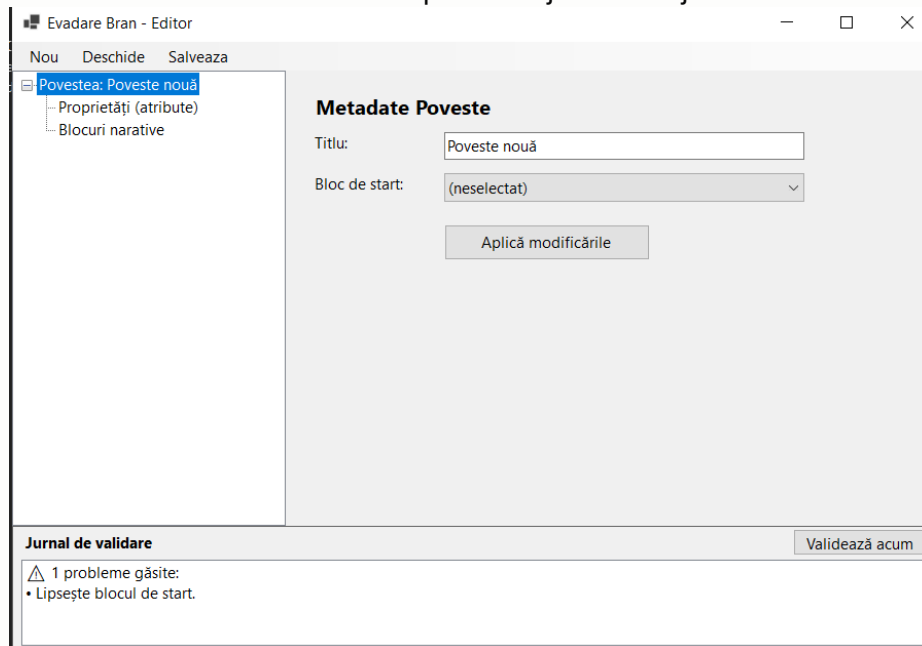


Figura 7.9 — Fereastra principală a Editorului la pornire.

7.2.2. Selectând rădăcina arborelui se deschide editorul de metadata, unde se stabilesc titlul poveștii și blocul de start (ales dintr-o listă derulantă cu blocurile existente).

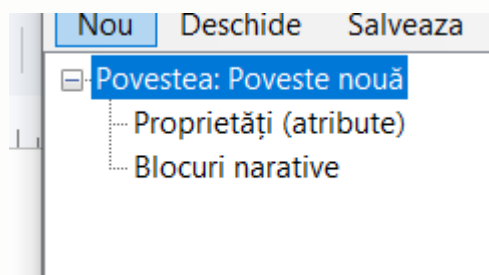


Figura 7.10 — Editarea metadatelor poveștii.

7.2.3. Categoria „Proprietăți (atribute)” permite adăugarea de atribute noi și afișează lista celor existente, cu valorile lor (min, max, inițial).

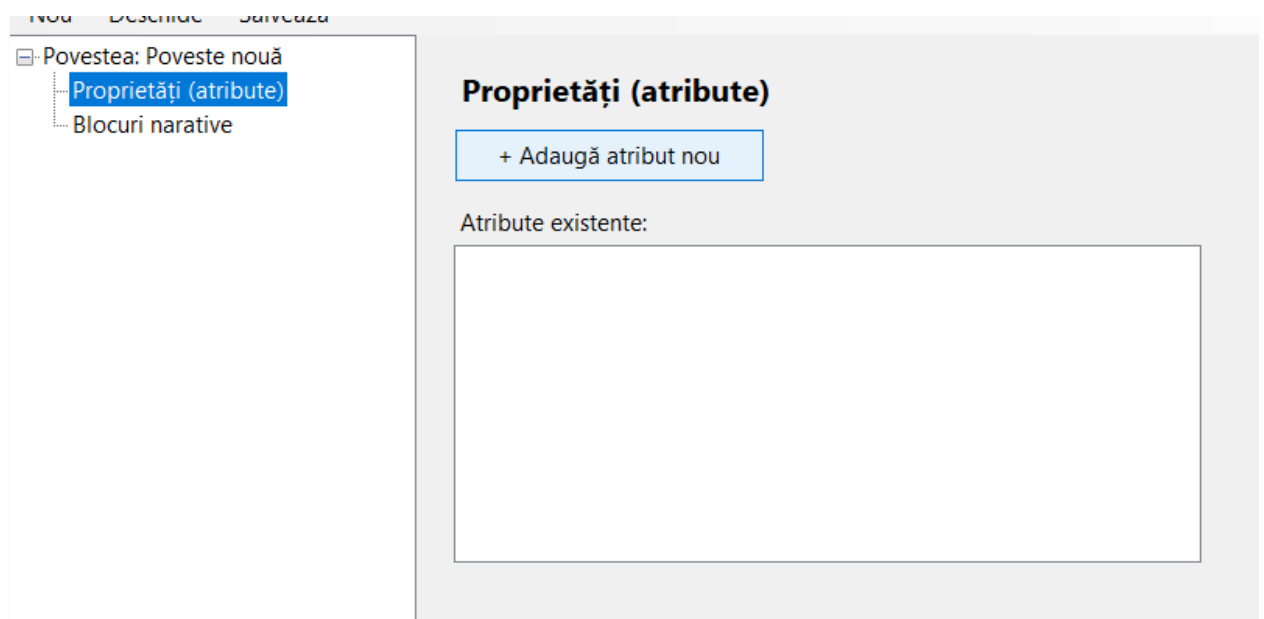


Figura 7.11 — Lista atributelor poveștii.

7.2.4. La selectarea unui atribut se deschide editorul complet: cheia, eticheta din HUD, valorile minim/maxim/inițial, vizibilitatea în HUD, ordinea de afișare și blocurile de redirecționare la atingerea limitei minime sau maxime. La aplicare se verifică automat că $\text{Min} \leq \text{Inițial} \leq \text{Max}$.

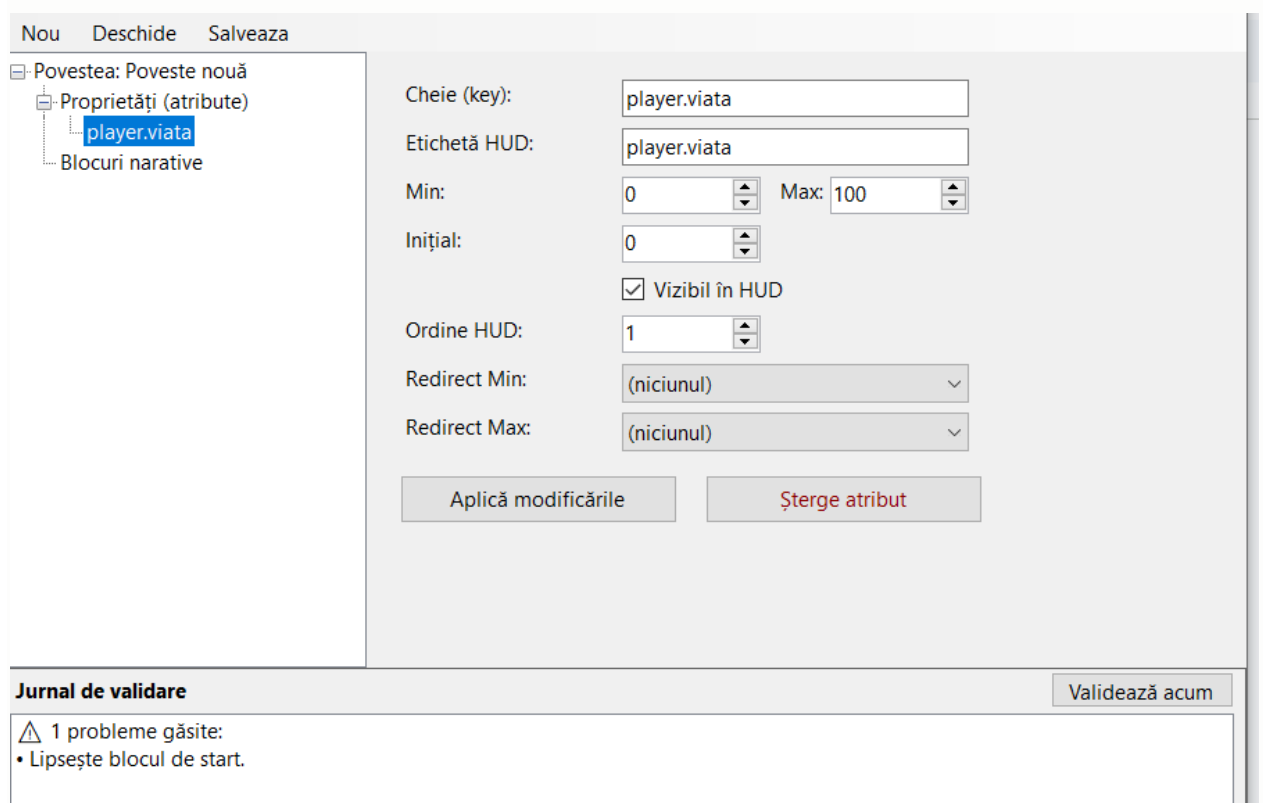


Figura 7.12 — Editorul complet al unui atribut.

7.2.5. Categoria „Blocuri narative” permite adăugarea de blocuri noi și afișează lista lor, marcând blocurile finale și numărul de decizii al fiecăruia.

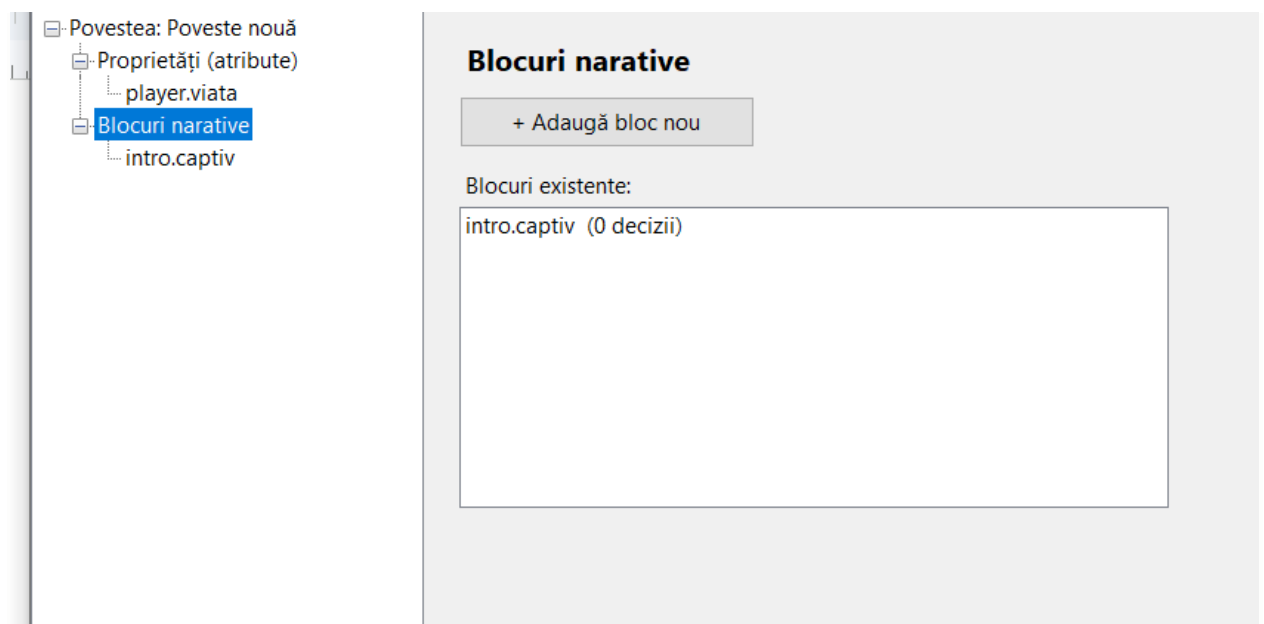


Figura 7.13 – Lista blocurilor narative.

7.2.6. Editorul unui bloc cuprinde textul narativ, opțiunea de bloc final, imaginea asociată (cu previzualizare și buton de răsfoire) și lista deciziilor blocului, cu butoane de adăugare, editare și ștergere.

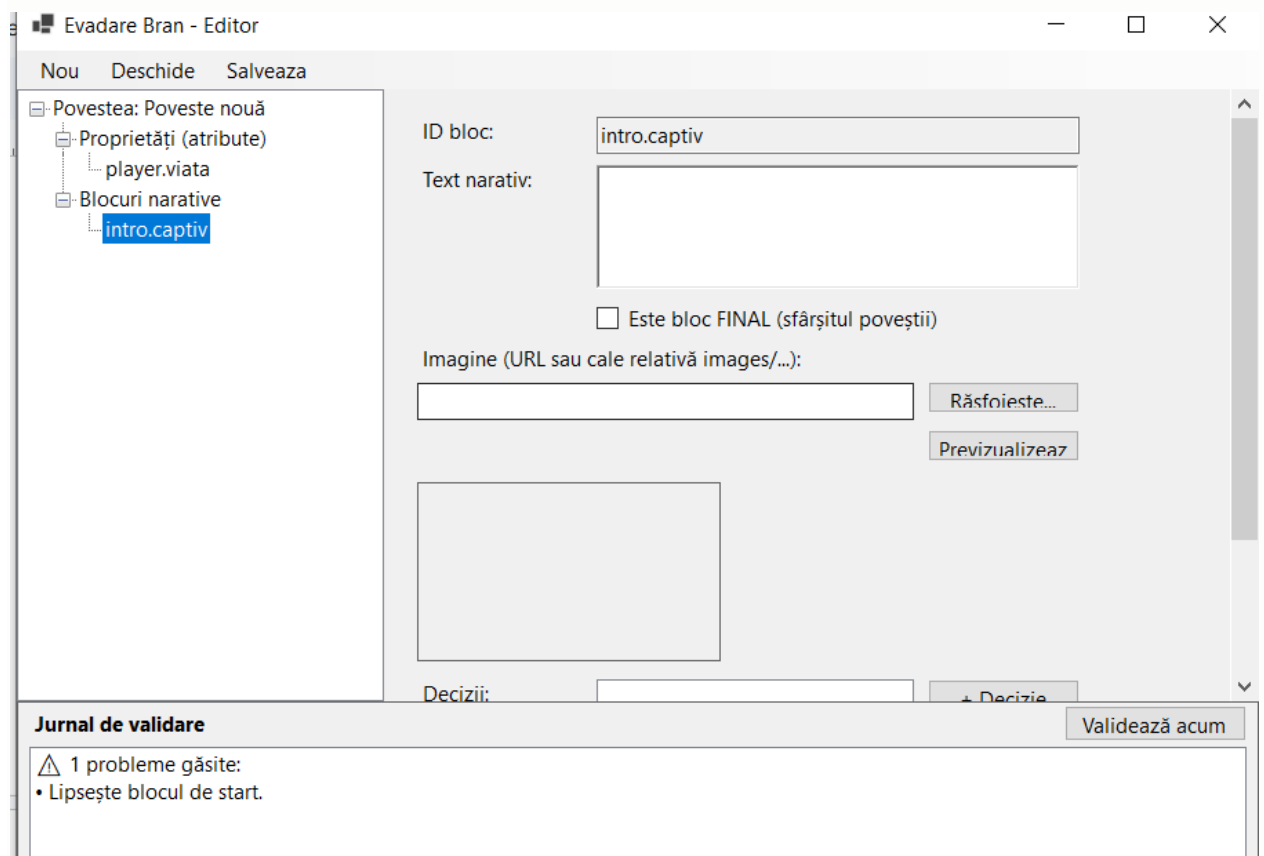


Figura 7.14 – Editorul complet al unui bloc narativ.

7.2.7. Adăugarea sau editarea unei decizii se face printr-o fereastră dedicată, unde se stabilesc textul deciziei, blocul țintă, efectele asupra atributelor (de tip ADD sau SET) și, opțional, condiția care determină afișarea deciziei.

Textul deciziei:

Blocul destinație:

Iconiță (URL sau cale):

Răsfoiește...

Editează condiția de afișare...

Efekte (tip ADD/SET, atribut, valoare):

	Tip	Atribut	Valoare
*			

Salvează decizia

Figura 7.15 – Fereastra de editare a unei decizii.

7.2.8. La salvare, povestea este împachetată într-o arhivă .zip ce conține fișierul `story.json` și folderul `images/` cu imaginile referite. Un mesaj confirmă salvarea reușită.

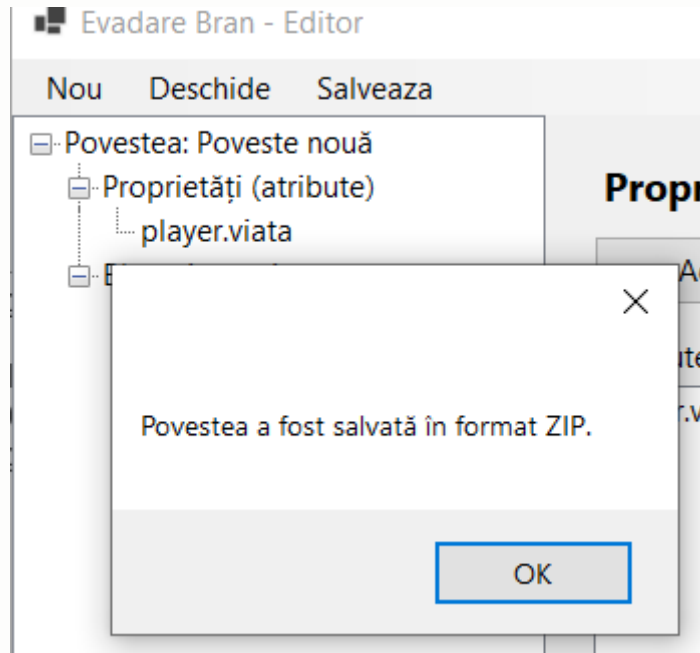


Figura 7.16 – Salvarea poveștii ca pachet ZIP.

7.3. Validarea poveștii

7.3.1. Jurnalul de validare din partea de jos a Editorului se reîmprospătează automat la fiecare modificare a structurii. El semnalează problemele găsite: titlu sau bloc de start lipsă, ID-uri sau chei duplicate, decizii care ținesc blocuri inexistente, efecte/condiții pe atribute inexistente, imagini lipsă și blocuri inaccesibile.

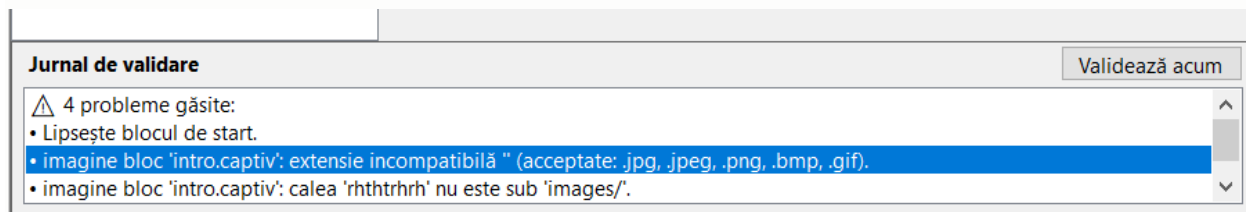


Figura 7.17 – Jurnalul de validare semnalând probleme.

8. Concluzii

Proiectul **Evadare din Castelul Bran** demonstrează modul în care principiile fundamentale ale programării orientate pe obiecte pot fi aplicate într-un sistem software real, cu arhitectură pe straturi, distribuit pe trei componente independente. Echipa de patru persoane a abordat proiectul pornind de la o separare clară a responsabilităților, ceea ce a permis dezvoltarea paralelă și integrarea fără conflicte structurale.

Moroșanu Răzvan a proiectat și implementat biblioteca `Story.Core`, care reprezintă fundația întregului sistem. Deciziile de design adoptate — indexarea blocurilor printr-un `Dictionary<string, BlocPoveste>` pentru acces în timp constant $O(1)$, gestionarea istoricului de navigare printr-o stivă (`Stack<string>`), evaluarea recursivă a arborilor sintactici abstracți (AST) și formula de clampare matematică `Math.Max (Min, Math.Min (Max, val))` — reflectă o înțelegere solidă a structurilor de date și a algoritmilor de bază. Biblioteca respectă principiul *deschis/închis*: extinderea sistemului cu noi tipuri de efecte sau condiții nu necesită modificarea aplicațiilor consumatoare.

Panainte Bogdan a construit aplicația player `EvadareBranReader`, respectând o separare strictă între logica de joc (delegată complet motorului) și stratul de prezentare. Soluțiile tehnice implementate — gestionarea folderelor temporare cu identificatori GUID, încărcarea asincronă a imaginilor de pe URL-uri externe prin `PictureBox.LoadAsync()`, eliberarea corectă a resurselor prin blocuri `using` și generarea dinamică a butoanelor de decizie filtrate prin motorul AST — reflectă bune practici WinForms și o înțelegere profundă a ciclului de viață al aplicației.

Neculcea Sabin a proiectat infrastructura editorului, optând pentru o arhitectură cu o singură fereastră reactivă în locul multiplelor dialoguri modale — o decizie care reduce consumul de memorie și simplifică fluxul de lucru al autorului. Mecanismul de rutare bazat pe proprietatea `Tag`

a nodului selectat din `TreeView`, generarea procedurală a controalelor UI și sistemul de “tracking” al imaginilor noi prin `Dictionary` — cu rezolvare finală prin `CopiazăImaginiReferite()` — demonstrează o abordare inginerescă riguroasă a persistenței și distribuției conținutului.

Pricop Andrei a adus cel mai complex subsistem al editorului: editorul vizual de arbori logici (AST) cu serializare/deserializare recursivă, sistemul de validare live și algoritmul BFS pentru detectarea blocurilor inaccesibile. Aceste componente transformă editorul dintr-un simplu formular de introducere a datelor într-un instrument de authoring cu asistență activă, garantând integritatea structurală a poveștii înainte ca aceasta să ajungă la jucător.

În ansamblu, proiectul demonstrează aplicarea consecventă a principiilor POO: *încapsulare* (setter-e private pe proprietățile motorului), *compoziție* (ierarhia de clase din modelul de date), *separarea responsabilităților* (trei straturi arhitecturale distincte) și *reutilizarea codului* (biblioteca comună folosită de ambele aplicații). Formatul de schimb de date bazat pe JSON și arhivele ZIP asigură portabilitatea poveștilor între orice instanțe ale aplicației, fără dependențe de mediu sau bază de date.

Prin complexitatea arhitecturală — bibliotecă comună partajată, două aplicații WinForms distincte, sistem de validare live, evaluator recursiv de condiții și format de date portabil — echipa a depășit cerințele minime ale temei, livrând un sistem complet și consistent, fără duplicare de cod și cu un grad ridicat de extensibilitate pentru eventuale dezvoltări ulterioare.

9. Bibliografie

Documentatie oficială Microsoft

- [1] Microsoft Corporation. (2024). *C# Language Reference — Classes, Structs and Records*. Microsoft Learn. Disponibil la: <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/> [Accesat: mai 2026]
- [2] Microsoft Corporation. (2024). *System.Text.Json — How to serialize and deserialize JSON in .NET*. Microsoft Learn. Disponibil la: <https://learn.microsoft.com/en-us/dotnet/standard/serialization/system-text-json/> [Accesat: mai 2026]
- [3] Microsoft Corporation. (2024). *Windows Forms for .NET — Overview and application development*. Microsoft Learn. Disponibil la: <https://learn.microsoft.com/en-us/dotnet/desktop/winforms/> [Accesat: mai 2026]
- [4] Microsoft Corporation. (2024). *Dictionary<TKey, TValue> Class — System.Collections.Generic*. Microsoft Learn. Disponibil la: <https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic.dictionary-2> [Accesat: mai 2026]

[5] Microsoft Corporation. (2024). *Stack<T> Class* — *System.Collections.Generic*. Microsoft Learn. Disponibil la: <https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic.stack-1> [Accesat: mai 2026]

[6] Microsoft Corporation. (2024). *System.IO.Compression.ZipFile Class*. Microsoft Learn. Disponibil la: <https://learn.microsoft.com/en-us/dotnet/api/system.io.compression.zipfile> [Accesat: mai 2026]

Cărți de referință

[7] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional. ISBN: 978-0-201-63361-0

[8] Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall. ISBN: 978-0-13-235088-4

[9] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms*, ediția a 3-a. MIT Press. ISBN: 978-0-262-03384-8

[10] Albahari, J., & Albahari, B. (2022). *C# 10 in a Nutshell: The Definitive Reference*. O'Reilly Media. ISBN: 978-1-098-12195-2

[11] Troelsen, A., & Japikse, P. (2022). *Pro C# 10 with .NET 6: Foundational Principles and Practices in Programming*, ediția a 10-a. Apress. ISBN: 978-1-4842-7869-8

Standarde și specificații tehnice

[12] ECMA International. (2022). *ECMA-334: C# Language Specification*, ediția a 6-a. ECMA International. Disponibil la: <https://www.ecma-international.org/publications-and-standards/standards/ecma-334/>

[13] Bray, T. (Ed.). (2017). *RFC 8259: The JavaScript Object Notation (JSON) Data Interchange Format*. Internet Engineering Task Force (IETF). Disponibil la: <https://datatracker.ietf.org/doc/html/rfc8259>

Resurse web și cursuri academice

[14] Departamentul Automatică și Tehnologia Informației. (2026). *Cerințe temă de semestru POO/PCLP3 — Motor de poveste interactivă*. Universitatea Transilvania din Brașov, Facultatea de Inginerie Electrică și Știința Calculatoarelor.

[15] Martin, R. C. (2000). *Design Principles and Design Patterns*. Object Mentor. Disponibil la: https://staff.cs.utu.fi/staff/jouni.smed/doos_06/material/DesignPrinciplesAndPatterns.pdf [Accesat: mai 2026]

[16] Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional. ISBN: 978-0-321-12742-6

[17] Knuth, D. E. (1997). *The Art of Computer Programming, Vol. 1: Fundamental Algorithms*, ediția a 3-a. Addison-Wesley Professional. ISBN: 978-0-201-89683-1

[18] Microsoft Corporation. (2024). *.NET 10 Runtime and SDK — What's new*. Microsoft Learn. Disponibil la: <https://learn.microsoft.com/en-us/dotnet/core/whats-new/dotnet-10/overview> [Accesat: mai 2026]